1. O "Muro da Confusão" e Arquitetura de Software — Explique o conceito do "Muro da Confusão" no contexto de projetos de Ciência de Dados e Engenharia de Software. Em seguida, disserte sobre como a adoção de uma arquitetura de microsserviços ajuda a superar esse obstáculo corporativo em comparação com sistemas baseados em arquitetura monolítica.

R: O "Muro da Confusão" é a barreira técnica que separa Ciência de Dados de Engenharia de Software. Na ciência de dados se faz uso de notebooks, priorizando resultados rápidos, enquanto engenheiros precisam de código testável, versionado, escalável e implantável. O ‘muro’ surge quando o modelo desenvolvido no notebook não consegue ser integrado ao sistema produtivo, gerando retrabalho e conflitos entre as equipes. A arquitetura de microsserviços ajuda a superar esse obstáculo porque permite que cada parte do sistema seja desenvolvida, testada e implantada de forma independente. Assim, o serviço de inferência de Machine Learning pode ser mantido pela equipe de dados em Python, enquanto a API que recebe as requisições fica com a equipe de engenharia em Java. Essa separação força a definição de contratos claros (APIs REST, por exemplo), reduz ambiguidades e permite que cada serviço evolua sem quebrar o outro. Em um monolito, todas as responsabilidades ficam concentradas em um único artefato, então qualquer mudança no modelo de ML exige rebuild e redeploy da aplicação inteira, o que agrava o "Muro da Confusão" e torna a colaboração entre as equipes muito mais difícil.

2. Padrões de Projeto e Separação de Responsabilidades — Em ambientes de experimentação de Ciência de Dados, é comum o uso de Jupyter Notebooks onde a conexão com o banco de dados, o tratamento dos dados e a inferência do modelo ocorrem frequentemente no mesmo bloco de código. Contraste essa prática com a Arquitetura de 3 Camadas (Controller, Service e Repository) ensinada no Spring Boot, destacando os benefícios dessa separação estrutural para a estabilidade do sistema.

R: No Jupyter Notebook, é comum que conexão com banco, tratamento de dados e inferência fiquem na mesma célula. Isso funciona para experimentação, mas é ruim para produção, porque mistura responsabilidades, dificulta testes, torna

o código frágil e impede o reuso. A Arquitetura de 3 Camadas do Spring Boot resolve isso separando o Controller (que recebe a requisição HTTP e devolve a resposta), o Service (que contém as regras de negócio e a orquestração) e o Repository (que acessa o banco de dados). Com essa separação, uma mudança no banco afeta apenas o Repository; é possível testar o Service sem subir o banco, usando mocks; o mesmo Service pode ser reaproveitado por diferentes Controllers; e a lógica de negócio fica isolada de detalhes de infraestrutura. Isso torna o sistema muito mais estável e fácil de manter, pois cada camada pode evoluir de forma independente. O que funciona no notebook como protótipo precisa ser refatorado para essas camadas antes de ir para produção.

3. Inversão de Controle (IoC) e Injeção de Dependência — O Spring Framework baseia-se fortemente no conceito de Inversão de Controle (IoC). Descreva o que significa esse conceito e explique a diferença prática, em termos de acoplamento e gerenciamento de memória, entre instanciar objetos manualmente (usando a palavra-chave new a cada requisição) e utilizar a Injeção de Dependência do Spring (por exemplo, via construtor ou @Autowired).

R: A Inversão de Controle (IoC) significa que o controle da criação e do gerenciamento dos objetos sai do código da aplicação e passa para um *container*, que no caso é o *Spring*. Em vez da classe criar suas dependências com *new*, ela as recebe prontas. Na prática, instanciar objetos manualmente a cada requisição gera alto acoplamento, porque a classe precisa conhecer a implementação concreta criando um objeto novo toda vez e aumentando a pressão no *garbage collector*. Isso dificulta os testes, pois não dá para substituir por um *mock* sem alterar o código, deixando o ciclo de vida descontrolado. Já com a Injeção de Dependência do *Spring*, o código depende de interfaces, o que reduz o acoplamento, os *beans* são *singletons* por padrão, ou seja, criados uma vez e reutilizados, o que é muito mais eficiente em memória, fácil de testar, bastando passar um *mock* no construtor e o *container* gerencia todo o ciclo de vida dos objetos. Por isso, IoC e DI tornam o código mais limpo, testável e eficiente.

4. Contratos de Interface e DTOs — No desenvolvimento de APIs RESTful, a documentação e a estruturação de dados são essenciais. Descreva a função

dos Data Transfer Objects (DTOs) na serialização/deserialização de JSONs e explique como a implementação do Swagger/OpenAPI atua na resolução do problema conhecido como "API Caixa-Preta" no ecossistema de microsserviços.

R: Os *DTOs (Data Transfer Objects)* são objetos simples que transportam dados entre camadas ou sistemas. Em *APIs RESTful*, eles servem para desacoplar o modelo de domínio da representação externa em *JSON*, definir exatamente quais campos são expostos (evitando vazar dados sensíveis como senhas), permitir versionamento sem quebrar o contrato e facilitar validações com anotações. Já o *Swagger/OpenAPI* resolve o problema da falta de informação da *API*, que acontece quando os consumidores não sabem quais *endpoints* existem, quais parâmetros enviar ou qual o formato da resposta. Com *OpenAPI*, a documentação é gerada automaticamente a partir do código, há uma interface interativa (*Swagger UI*) para testar os *endpoints*, os contratos ficam explícitos e versionados e é possível gerar clientes automaticamente. Em microsserviços, isso é essencial, porque sem contrato claro cada equipe teria que adivinhar como consumir o serviço da outra, o que aumentaria erros e retrabalho.

5. Integração de Sistemas e Processos (Java + Python) — Para consumir o modelo de Machine Learning (Regressão Logística com LBP) através do Java, foi utilizada a classe ProcessBuilder. Explique detalhadamente como o Java atua como um "Orquestrador" nesse cenário de integração via Sistema Operacional. Quais são as etapas que o Java realiza desde o recebimento da requisição HTTP até a devolução do resultado gerado pelo script Python?

R: O Java atua como orquestrador ao usar *ProcessBuilder* para invocar um *script Python* que contém o modelo de ML. O fluxo é o seguinte: o *Controller* recebe a requisição *HTTP* com a imagem via *MultipartFile*, o *Java* salva a imagem em disco com *Files.createTempFile*, porque o *script Python* precisa de um caminho de arquivo acessível. O *ProcessBuilder* é configurado com o executável *Python* e com o caminho do *script* e da imagem. O *process.start()* cria um processo filho no sistema operacional e o Java aguarda com *process.waitFor()*, o *Java* lê o *stdout* do *Python* (que retorna a classificação, como "1" ou "0") e o *stderr* para *logs* e erros. O resultado é convertido em um

DTO de resposta, o arquivo temporário é removido com *Files.deleteIfExists* e o *Controller* devolve o *JSON* com o resultado.

É importante tratar *timeouts*, códigos de saída diferentes de zero e erros de *parsing*. Além disso, como o processo *Python* é *stateless*, cada requisição carrega o modelo novamente, o que pode ser lento, uma alternativa é manter um serviço *Python* persistente, como *Flask* ou *FastAPI*, para evitar esse custo.

6. Manipulação de Arquivos e Saúde do Servidor — Ao desenvolver o endpoint para receber imagens da esteira da fábrica (MultipartFile), a aplicação salva temporariamente as imagens utilizando métodos como Files.createTempFile e, posteriormente, as exclui (Files.deleteIfExists). Justifique por que não se deve salvar as imagens utilizando seus nomes originais de envio e analise a importância da exclusão do arquivo temporário para a "higiene" e resiliência do servidor.

R: Não se deve salvar as imagens com o nome original de envio porque dois arquivos com o mesmo nome se sobrescreveriam, nomes originais podem conter *path traversal* (como ../../etc/passwd) ou caracteres maliciosos, representando risco de segurança, e o nome original pode expor informações do cliente. Usar nomes gerados pelo sistema, como *UUID* mais *timestamp*, evita conflitos e facilita a auditoria. A exclusão do arquivo temporário é fundamental para a higiene e resiliência do servidor, porque sem ela o disco enche rapidamente, especialmente em esteiras industriais que enviam imagens continuamente. Disco cheio derruba a aplicação, afeta outros serviços e pode causar perda de dados. Além disso, imagens temporárias podem conter dados sensíveis, e deixá-las no disco é um risco de vazamento e de não conformidade com a LGPD. Por isso, a exclusão deve ser garantida com *try/finally* ou t*ry-with-resources*, para que ocorra mesmo em caso de exceção.

7. Containerização vs. Máquinas Virtuais (VMs) — Um dos maiores problemas no deploy de aplicações de Ciência de Dados é a síndrome do "Funciona na minha máquina". Defina o que é a containerização (utilizando o Docker como referência) e diferencie-a da virtualização tradicional (Máquinas

Virtuais), destacando as vantagens do Docker em relação ao compartilhamento do núcleo (kernel) do sistema operacional e velocidade de inicialização.

R: A *containerização* com *Docker* empacota a aplicação e suas dependências em uma imagem que roda de forma isolada sobre o *kernel* do sistema operacional. Já a virtualização tradicional (VMs) emula hardware completo e exige um *hypervisor* mais um sistema operacional convidado por instância. A grande diferença é que *containers* compartilham o *kernel* do *host*, enquanto cada VM tem seu próprio *kernel*. Isso faz com que *containers* sejam muito menores (MBs contra GBs), iniciem em segundos (contra minutos das VMs) e permitam alta densidade no mesmo hardware. O isolamento do *container* é feito em nível de processo, com *namespaces* e *cgroups*, enquanto o da VM é em nível de hardware. A principal vantagem do *Docker* é justamente o compartilhamento do *kernel*, que elimina o *overhead* do sistema operacional convidado, e a velocidade de inicialização, que o torna ideal para CI/CD e escalabilidade horizontal. A imagem também congela as dependências, resolvendo o "funciona na minha máquina". A limitação é que o isolamento é menos forte que o de uma VM, o que exige cuidados extras de segurança.

8. Anti-Padrões em Imagens Docker — Durante a construção do Dockerfile do projeto, foi gerada uma imagem que continha, simultaneamente, o ecossistema do Java (JVM), o ecossistema do Python e as bibliotecas de Machine Learning (como scikit-learn e opencv). Discuta por que ter uma imagem pesada (ex: 1.5 GB) executando múltiplos ambientes diferentes é considerado um "anti-padrão" no design de microsserviços, e sugira conceitualmente uma abordagem arquitetural mais refinada para resolver essa questão.

R: Uma imagem de 1.5 GB com *JVM, Python, scikit-learn* e *OpenCV* é um anti-padrão porque viola o princípio da responsabilidade única: um *container* deveria executar apenas uma função, e nesse caso misturar *Java* e *Python* significa que o *docker* faz coisas demais. Além disso, a imagem pesada aumenta o tempo de *build, push, pull* e *deploy*, amplia a superfície de ataque com mais bibliotecas e vulnerabilidades, acopla os ciclos de vida, de modo que atualizar o *scikit-learn* exige *rebuild* da aplicação *Java* inteira, impede escalar apenas a parte de ML,

desperdiça recursos, com a *JVM* ociosa enquanto o *Python* processa e quebra o cache do *Docker*, tornando os *builds* lentos.

A solução é separar em dois microsserviços: um serviço *Java* (*API*/Orquestrador), que recebe *HTTP*, valida e chama o serviço de ML e um serviço Python, que expõe uma *API* com *FastAPI* ou *Flask*, carrega o modelo uma vez e responde às inferências. A comunicação entre eles pode ser via *HTTP/REST* ou *gRPC*, com contrato *OpenAPI*, daí a importância de documentar as *API's*